

DOCUMENTO DE PROYECTO — VERSION 2.0

SCV — Sistema de Control Vehicular

Aplicación Web Progresiva (PWA)

Campo	Detalle
Empresa	Comercializadora Normetales S.A.S
Autor	Area de Desarrollo de Software
Versión	2.0 — Modelo BD revisado, roles y permisos definidos
Fecha	9 de abril de 2026
Estado	En planificación

1. Introducción

Este documento describe el diseño, alcance, metodología y hoja de ruta del Sistema de Control Vehicular (SCV), una aplicación web progresiva (PWA) que digitalizara los procesos de registro de movimientos de vehículos y la lista de chequeo preoperacional de la empresa.

Esta es la versión 2.0 del documento e incorpora los siguientes cambios respecto a la versión anterior: modelo de base de datos revisado con separación completa entre movimientos y chequeos; definición de tres roles de usuario con permisos diferenciados; y distinción explícita entre usuarios del sistema y el catálogo de conductores.

2. Contexto y Problemática

2.1 Situación actual

- Registro de entrada y salida de vehículos mediante formulario de Google Forms.
- Lista de chequeo preoperacional en papel, sin respaldo digital ni historial.
- Copias de seguridad físicas almacenadas en un servidor NAS.
- Sin sistema de gestión interno — todo en papel o Excel.

2.2 Problemas identificados

#	Problema	Impacto
1	Datos de vehículos dispersos en Google Sheets	Difícil trazabilidad y consulta
2	Chequeos preo en papel se pierden o no se hacen	Riesgo legal y de seguridad vial
3	Sin alertas ni notificaciones	Vehículos salen sin chequeo completo
4	Sin reportes históricos por vehículo o conductor	Imposible auditar ni analizar
5	Dependencia de Google y conexión a internet	Vulnerabilidad ante caídas del servicio

3. Usuarios del Sistema y Permisos

3.1 Concepto clave: usuarios vs conductores

El sistema maneja dos conceptos que NO deben confundirse:

Concepto	Que es	¿Tiene login?	Ejemplo
Usuario	Persona que opera el sistema desde la app	Si	El operario que registra entradas/salidas en portería
Conductor	Persona que maneja los vehículos de la empresa	No	Pedro, conductor del camión ABC-123

La tabla de conductores es un catálogo de referencia, igual que la tabla de vehículos. Almacena todos los conductores de la empresa con sus datos (cedula, licencia, categoría). Un conductor se selecciona en los formularios, pero nunca inicia sesión en el sistema.

3.2 Roles de usuario

Rol	Quien lo usa	Descripción
admin	Jefe / coordinador logístico	Acceso total al sistema
operario_movimientos	Personal de portería / despacho	Solo registra entradas y salidas de vehículos
operario_chequeo	Mecánico / inspector de vehículos	Solo llena la lista de chequeo preoperacional

3.3 Matriz de permisos

Funcionalidad	admin	op. movimientos	op. chequeo
Registrar movimiento (entrada/salida)	Si	Si	No
Ver historial de movimientos	Si	Si	No
Registrar chequeo preoperacional	Si	No	Si
Ver historial de chequeos	Si	No	Si
Gestionar vehículos (CRUD)	Si	No	No
Gestionar conductores (CRUD)	Si	No	No
Gestionar usuarios (CRUD)	Si	No	No
Ver dashboard y reportes	Si	No	No

Funcionalidad	admin	op. movimientos	op. chequeo
Exportar a Excel	Si	No	No

4. Arquitectura del Sistema

4.1 Stack tecnológico

Capa	Tecnología	Responsabilidad
Frontend (PWA)	HTML + CSS + JavaScript	Interfaz mobile-first, instalable como app
Backend (API REST)	Python + FastAPI	Lógica de negocio, autenticación JWT, endpoints
Base de datos	SQLite (MVP) -> PostgreSQL	Persistencia, consultas, reportes
Infraestructura	PC / servidor local de la empresa	Hospedaje en red interna sin dependencia de internet

Todos los usuarios acceden desde su celular o PC a través de la red WiFi de la empresa. El backend corre en un computador que permanece encendido. No se requiere internet externo para el funcionamiento diario.

5. Modelo de Base de Datos

5.1 Principios de diseño

- La tabla movimientos y la tabla chequeos son completamente independientes. No existe llave foránea entre ellas.
- Ambas tablas se vinculan de forma independiente a vehículos y conductores.
- La tabla usuarios es independiente de la tabla conductores. Un usuario opera el sistema; un conductor maneja el vehículo.
- La tabla chequeo_items almacena cada ítem del formulario como una fila individual.

5.2 Descripción de las tablas

Tabla: usuarios

Personas que tienen acceso al sistema. Son completamente independientes de los conductores. Su rol define que pueden hacer.

Campo	Tipo	Descripción
id	INTEGER PK	Identificador único autoincremental
nombre	VARCHAR	Nombre completo del usuario
email	VARCHAR	Correo para iniciar sesión (único)
password_hash	VARCHAR	Contraseña encriptada con bcrypt
rol	VARCHAR	admin operario_movimientos operario_chequeo
activo	BOOLEAN	Permite desactivar sin borrar el registro
fecha_creacion	TIMESTAMP	Fecha en que fue creado el usuario

Tabla: vehículos

Catálogo de todos los vehículos de la empresa. Tabla de referencia independiente.

Campo	Tipo	Descripción
id	INTEGER PK	Identificador único autoincremental
placa	VARCHAR	Placa del vehículo (única)
marca	VARCHAR	Marca del vehículo
modelo	VARCHAR	Modelo del vehículo
año	INTEGER	Año de fabricación
empresa	VARCHAR	Empresa propietaria o contratante
activo	BOOLEAN	Permite dar de baja sin borrar historial

Tabla: conductores

Catálogo de todos los conductores de la empresa. Tabla de referencia independiente. Los conductores no tienen acceso al sistema — solo se seleccionan en los formularios de movimientos y chequeos.

Campo	Tipo	Descripción
id	INTEGER PK	Identificador único autoincremental
nombre	VARCHAR	Nombre completo del conductor
cedula	VARCHAR	Numero de cedula (único)
licencia	VARCHAR	Numero de licencia de conducción
categoría	VARCHAR	Categoría de la licencia (C1, C2, etc.)
activo	BOOLEAN	Permite dar de baja sin borrar historial

Tabla: movimientos

Registra cada entrada o salida de un vehículo. Es completamente independiente de chequeos. Se vincula a vehículos, conductores y usuarios (quien registro el movimiento) mediante llaves foráneas.

Campo	Tipo	Descripción
id	INTEGER PK	Identificador único autoincremental
tipo	VARCHAR	salida entrada
vehiculo_id	INTEGER FK	Llave foránea -> vehiculos.id
conductor_id	INTEGER FK	Llave foránea -> conductores.id
usuario_id	INTEGER FK	Llave foránea -> usuarios.id (operario que registro)
auxiliar	VARCHAR	Nombre del auxiliar (opcional)
proveedor	VARCHAR	Destino o proveedor (opcional)
kilometraje	INTEGER	Kilometraje al momento del registro
bascula	DECIMAL	Peso registrado en bascula en kg
sacas	INTEGER	Cantidad de sacas de material
cajón	VARCHAR	Observación sobre el cajón del vehículo
observaciones	TEXT	Observaciones adicionales (opcional)
fecha_hora	TIMESTAMP	Fecha y hora exacta del registro (automática)

Tabla: chequeos

Registra la cabecera del chequeo preoperacional. Completamente independiente de movimientos. Se vincula a vehículos, conductores y usuarios (quien lleno el chequeo) de forma independiente.

Campo	Tipo	Descripción
id	INTEGER PK	Identificador único autoincremental
vehiculo_id	INTEGER FK	Llave foránea -> vehiculos.id
conductor_id	INTEGER FK	Llave foránea -> conductores.id
usuario_id	INTEGER FK	Llave foránea -> usuarios.id (operario que lleno el chequeo)
kilometraje	INTEGER	Kilometraje al momento del chequeo
fecha_venc_soat	DATE	Fecha de vencimiento del SOAT
fecha_venc_rtm	DATE	Fecha de vencimiento de la RTM
fecha_venc_extintor	DATE	Fecha de vencimiento del extintor
obs_generales	TEXT	Observaciones generales del chequeo (opcional)

Campo	Tipo	Descripción
fecha_hora	TIMESTAMP	Fecha y hora exacta del registro (automática)

Tabla: chequeo_items

Almacena cada ítem inspeccionado como una fila independiente. Un solo chequeo genera aproximadamente 45 filas en esta tabla, una por cada ítem del formulario real.

Campo	Tipo	Descripción
id	INTEGER PK	Identificador único autoincremental
chequeo_id	INTEGER FK	Llave foránea -> chequeos.id
sección	VARCHAR	Sección del formulario: frenos luces cabina llantas etc.
ítem	VARCHAR	Ítem inspeccionado: freno_principal luces_altas etc.
valor	VARCHAR	Resultado: ajustado largo conforme no_conforme full medio etc.
observación	TEXT	Observación específica de este ítem (opcional)

6. Relaciones entre Tablas

6.1 Tabla de relaciones

Tabla origen	Relación	Tabla destino	Significado
vehículos	1 a muchos	movimientos	Un vehículo aparece en muchos movimientos. Cada movimiento tiene un solo vehículo.
conductores	1 a muchos	movimientos	Un conductor aparece en muchos movimientos. Cada movimiento tiene un solo conductor.
usuarios	1 a muchos	movimientos	Un operario_movimientos registra muchos movimientos. Cada movimiento fue registrado por un solo usuario.
vehículos	1 a muchos	chequeos	Un vehículo tiene muchos chequeos históricos. Cada chequeo corresponde a un solo vehículo.
conductores	1 a muchos	chequeos	Un conductor tiene muchos chequeos registrados. Cada chequeo referencia a un solo conductor.
usuarios	1 a muchos	chequeos	Un operario_chequeo llena muchos chequeos. Cada chequeo fue llenado por un solo usuario.
chequeos	1 a muchos	chequeo_items	Un chequeo genera ~45 ítems. Cada ítem pertenece a un solo chequeo.

6.2 Lo que NO existe como relación directa

- movimientos NO tiene llave foránea hacia chequeos. Son tablas completamente separadas.
- chequeos NO tiene llave foránea hacia movimientos. Cada una existe de forma independiente.
- usuarios NO tiene llave foránea hacia conductores. Son conceptos distintos.
- conductores NO tiene llave foránea hacia usuarios. Un conductor puede no tener cuenta.

6.3 Como se cruzan lógicamente movimientos y chequeos

Aunque no hay llave foránea entre ellas, en la práctica ambas tablas comparten `vehiculo_id` y `conductor_id`. Desde el panel del administrador, es posible cruzar los registros del día para saber si un vehículo que salió (movimientos) tenía o no un chequeo reciente (chequeos).

Momento	Quien actúa	Que registra	Tabla
Antes de salir	operario_chequeo	Inspección completa del vehículo	chequeos + chequeo_items
Al momento de salir	operario_movimientos	Salida del vehículo	movimientos (tipo: salida)
Al regresar	operario_movimientos	Retorno del vehículo	movimientos (tipo: entrada)

7. Requisitos del Sistema

7.1 Requisitos funcionales

ID	Modulo	Descripción
RF-01	Autenticación	Login con email y contraseña, sesión con token JWT
RF-02	Autenticación	3 roles: admin, operario_movimientos, operario_chequeo
RF-03	Autenticación	Cada rol solo accede a las funciones que le corresponden
RF-04	Vehículos	Admin puede registrar, editar y desactivar vehículos
RF-05	Conductores	Admin puede registrar, editar y desactivar conductores
RF-06	Usuarios	Admin puede crear, editar y desactivar cuentas de usuario
RF-07	Movimientos	operario_movimientos registra salida: vehículo, conductor, km, proveedor, bascula, sacas, cajón
RF-08	Movimientos	operario_movimientos registra entrada con los mismos campos
RF-09	Chequeo	operario_chequeo selecciona vehículo y conductor antes de llenar el chequeo
RF-10	Chequeo	El chequeo incluye todas las secciones del formulario real
RF-11	Chequeo	Cada sección permite observaciones independientes
RF-12	Reportes	Admin ve historial de movimientos filtrado por fecha, vehículo o conductor
RF-13	Reportes	Admin ve historial de chequeos filtrado por fecha, vehículo o conductor
RF-14	Exportación	Admin exporta reportes a Excel (.xlsx)
RF-15	PWA	La app es instalable en Android desde el navegador sin Play Store

7.2 Requisitos no funcionales

ID	Categoría	Descripción
RNF-01	Usabilidad	Interfaz móvil con botones grandes y flujo de máximo 3 pasos

ID	Categoría	Descripción
RNF-02	Seguridad	Contraseñas con hash bcrypt. Tokens JWT con expiración configurable
RNF-03	Seguridad	Cada endpoint valida el rol antes de ejecutar la acción
RNF-04	Disponibilidad	Funciona en red local sin depender de internet externo
RNF-05	Escalabilidad	Base de datos soporta al menos 5 años de datos sin degradarse
RNF-06	Mantenibilidad	Código documentado y versionado en GitHub con README detallado

8. MVP — Mínimo Producto Viable

8.1 Contenido del MVP

Modulo	Incluido en MVP	Queda para después
Autenticación	Login, 3 roles, JWT	Recuperación de contraseña
Vehículos	CRUD completo	Fotos, QR por vehículo
Conductores	CRUD completo	Alertas por vencimiento de licencia
Usuarios	CRUD completo por admin	Perfil editable por el propio usuario
Movimientos	Registro de salida y entrada con todos los campos	Mapa de recorrido
Chequeo pre-op	Formulario completo con todas las secciones reales	Fotos de evidencia por ítem
Reportes	Historial con filtros por fecha, vehículo, conductor	Gráficos, exportación PDF
PWA	Instalable en Android, diseño mobile-first	Modo offline completo, notif. push

8.2 Criterios de aceptación del MVP

1. Un operario_movimientos puede registrar una salida desde el celular en menos de 2 minutos.
2. Un operario_chequeo puede llenar el chequeo pre-operacional completo desde el celular.
3. El admin puede ver el historial de movimientos y chequeos desde el PC.
4. Los datos están almacenados en el servidor local de la empresa.
5. La app puede instalarse en un celular Android sin Play Store.
6. Cada rol solo puede acceder a las funciones que le corresponden.

9. Metodología de Desarrollo

9.1 Enfoque

Metodología ágil simplificada basada en Scrum para equipo de un solo desarrollador. Sprints de 1 semana con entregables funcionales al final de cada sprint.

9.2 Herramientas

Herramienta	Uso
GitHub	Control de versiones del código fuente
GitHub Issues / Projects	Seguimiento de tareas y bugs
Postman	Pruebas de la API REST
draw.io	Diagramas de flujo y modelo ER

9.3 Convenciones de código

- Python: snake_case. JavaScript: camelCase.
- Commits: feat: / fix: / docs: / refactor:
- Una rama por funcionalidad: feature/registro-movimientos, feature/chequeo-preop, etc.
- Rama main protegida — solo se fusiona código probado.

10. Roadmap de Desarrollo

Fase	Nombre	Duración	Entregable
Fase 0	Configuración inicial	2-3 días	Repos GitHub, entorno local, BD con las 6 tablas creadas
Fase 1	Backend + autenticación	1 semana	API con JWT, 3 roles, CRUD de vehículos, conductores y usuarios
Fase 2	Módulo de movimientos	1 semana	Endpoints y formulario móvil de salida y entrada
Fase 3	Módulo de chequeo pre-op	1 semana	Formulario móvil completo con todas las secciones del formulario real
Fase 4	Panel admin + reportes	1 semana	Dashboard, historial de movimientos y chequeos, exportación Excel
Fase 5	PWA + despliegue	3-4 días	App instalable en Android, despliegue en servidor local
Fase 6	Pruebas y ajustes	1 semana	Pruebas con usuarios reales, corrección de bugs

Tiempo estimado total del MVP: 6 a 7 semanas a tiempo parcial. A tiempo completo: 3 a 4 semanas.

11. Proyección y Trazabilidad Futura

Modulo futuro	Descripción	Prioridad
Mantenimiento vehicular	Registro de servicios, cambios de aceite, vencimiento de documentos	Alta
Alertas automáticas	Notificaciones por WhatsApp o email: documentos por vencer, ítems no conformes	Alta
Integración NAS	Respaldos automáticos de la BD en el NAS de la empresa	Alta
Foto de evidencia	Adjuntar fotos al chequeo o al movimiento	Media
Firma digital	El operario firma digitalmente el chequeo en pantalla táctil	Media
Geolocalización	Registrar coordenadas GPS al momento de la salida	Baja

12. Riesgos y Mitigación

Riesgo	Prob.	Impacto	Mitigación
Celulares Android muy antiguos no soportan PWA	Media	Alto	Probar en dispositivos reales en Fase 5
Operarios no adoptan la app	Media	Alto	Involucrar operarios en pruebas desde Fase 2
El servidor local falla o se apaga	Media	Alto	Documentar reinicio + respaldo en NAS
Cambios de requisitos a mitad del desarrollo	Alta	Medio	Metodología ágil + documentar cambios en GitHub
Desarrollador único sin reemplazo	Alta	Alto	Código documentado en GitHub con README detallado

13. Glosario

Termino	Definición
PWA	Progressive Web App — app web instalable en el celular que funciona como app nativa
API REST	Interfaz que permite al frontend comunicarse con el backend vía HTTP
FastAPI	Framework Python para crear APIs REST con documentación automática
JWT	JSON Web Token — sistema de autenticación con tokens seguros firmados digitalmente
SQLite	Base de datos en un solo archivo. Ideal para MVPs y proyectos pequeños
SQLAlchemy	Librería Python que traduce objetos a consultas SQL (ORM)
MVP	Mínimo Producto Viable — versión mínima que ya entrega valor real al negocio
NAS	Network Attached Storage — servidor de almacenamiento en red de la empresa
FK	Llave foránea — campo que apunta al ID de un registro en otra tabla
CRUD	Create, Read, Update, Delete — las 4 operaciones basicas sobre datos
Rol	Nivel de acceso de un usuario: define que puede ver y hacer en el sistema

14. Próximos Pasos Inmediatos

7. Crear los repositorios en GitHub: scv-backend y scv-frontend.
8. Instalar Python, FastAPI, SQLAlchemy y dependencias en el equipo de desarrollo.
9. Crear la base de datos SQLite con las 6 tablas definidas en la sección 5.
10. Implementar el endpoint de login con JWT y los 3 roles.
11. Crear el primer formulario móvil: registro de salida de vehículo.